Вернуться к 'Рекурсия. Функции высших порядков'

Тест начат	Пятница, 25 Сентябрь 2026, 23:11
Состояние	Завершенные
Завершен	Пятница, 25 Сентябрь 2026, 23:47
Прошло времени	36 мин. 29 сек.
Баллы	13,90/15,00
Оценка	**4,63** из 5,00 (**93**%)

Вопрос **1**

Верно

Баллов: 1,00 из 1,00

Выражение `foldr (-) 0 [1, 2, 3, 4]` равно

Выберите один ответ:

- `-2` ✔
- `-10`
- `0`
- `-8`
- `10`

Вопрос **2**

Верно

Баллов: 1,00 из 1,00

Используя следующее определение, покажите как вычисляется `init [1, 2, 3]`

```
init :: [a] -> [a]
init [_] = []
init (x : xs) = x : init xs
```

Выберите один ответ:

a.

```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : 2 : init [3]
= { applying init }
1 : 2 : 3 : init []
= { applying init }
1 : 2 : 3 : []
= { list notation }
[1,2,3]
```

b.

```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : []
= { list notation }
[1]
```

c.

```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : 2 : init [3]
= { applying init }
1 : 2 : []
= { list notation }
[1,2]
```

✔

d.

```
init [1,2,3]
= { applying init }
[1,2] : init [3]
= { applying init }
[1,2] : []
= { list notation }
[1,2]
```

Вопрос **3**

Верно

Баллов: 1,00 из 1,00

Выражение `takeWhile even [2, 4, 5, 6, 7, 8]` равно

Выберите один ответ:

- `[2,4,6]`
- `[]`
- `[2,4,6,8]`
- `[2]`
- `[2,4]` ✔

Вопрос **4**

Неверно

Баллов: -0,10 из 1,00

Выберите все варианты, которые реализуют функцию dropWhile

```
dropWhile :: (a -> Bool) -> [a] -> [a]
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

```
dropWhile _ [] = []
dropWhile p (x : xs)
  | p x = dropWhile p xs
  | otherwise = x : xs
```

```
dropWhile p = foldr (\ x acc -> if p x then acc else x : acc)
```

✖

```
dropWhile p = foldl add []
  where add [] x = if p x then [] else [x]
        add acc x = x : acc
```

```
dropWhile _ [] = []
dropWhile p (x : xs)
  | p x = dropWhile p xs
  | otherwise = xs
```

Вопрос **5**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию all

```
all :: (a -> Bool) -> [a] -> Bool
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один или несколько ответов:

- ☑ `all p xs = foldl (&&) True (map p xs)` ✔
- ☑ `all p = foldr (&&) True . map p` ✔
- ☐ `all p xs = map p (and xs)`
- ☑ `all p = and . map p` ✔
- ☐ `all p = map p . and`
- ☑ `all p = not . any (not . p)` ✔
- ☐ `all p xs = foldr (&&) False (map p xs)`
- ☑ `all p xs = and (map p xs)` ✔

Вопрос **6**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции слияния

```
merge :: Ord a => [a] -> [a] -> [a]
```

которая объединяет два списка, отсортированных в порядке возрастания в результирующий отсортированный в порядке возрастания список. Например:

```
> merge [2, 5, 6] [1, 3, 4]
[1, 2, 3, 4, 5, 6]
```

Выберите один ответ:

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then y : merge xs (y : ys) else x : merge (x : x
```

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then y : merge (x : xs) ys else x : merge xs (y
```

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then x : merge xs ys else y : merge xs ys
```

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then x : merge xs (y : ys) else y : merge (x : x
```

✔

Вопрос **7**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая выдает список с n одинаковыми элементами:

```
replicate :: Int -> a -> [a]
```

Выберите один ответ:

```
replicate 1 _ = []
replicate n x = replicate (n - 1) x ++ [x]
```

```
replicate 0 _ = []
replicate n x = x : replicate (n - 1) x
```

✔

```
replicate 0 _ = []
replicate n x = replicate (n - 1) x : x
```

```
replicate 1 x = x
replicate n x = x : replicate (n - 1) x
```

Вопрос **8**

Верно

Баллов: 1,00 из 1,00

Используя слдующее опрделеление, показать, как вычисляется

`drop 3 [1, 2, 3, 4, 5]`

```
drop :: Int -> [a] -> [a]
drop 0 xs = xs
drop n [] = []
drop n (_ : xs) = drop (n - 1) xs
```

Выберите один ответ:

a.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [2,3,4,5]
= { applying drop }
drop 1 [3,4,5]
= { applying drop }
drop 0 [4,5]
= { applying drop }
[4,5]
```

✔

b.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [1,2,4,5]
= { applying drop }
drop 1 [1,4,5]
= { applying drop }
drop 0 [4,5]
= { applying drop }
[4,5]
```

c.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [1,2,3,4]
= { applying drop }
drop 1 [1,2,3]
= { applying drop }
drop 0 [1,2]
= { applying drop }
[1,2]
```

d.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [2,3,4,5]
= { applying drop }
drop 1 [3,4,5]
= { applying drop }
drop 0 [4,5]
= { applying drop }
[5]
```

Вопрос **9**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию filter

```
filter :: (a -> Bool) -> [a] -> [a]
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

- `filter p = foldr (\ x xs -> if p x then xs ++ [x] else xs) []`
- `filter p = foldr (\ x xs -> if p x then x : xs else xs) []` ✔
- `filter p = foldl (\ x xs -> if p x then xs ++ [x] else xs) []`
- `filter p = foldl (\ xs x -> if p x then x : xs else xs) []`

Вопрос **10**

Верно

Баллов: 1,00 из 1,00

Выражение `filter even (map (+1) [1..5])` равно

Выберите один ответ:

- `[]`
- `[2,4]`
- `[2,4,6]` ✔
- `[3,5]`
- `[1,3,5]`

Вопрос **11**

Верно

Баллов: 1,00 из 1,00

Какие из предложенных выражений эквивалентны [1, 2, 3, 4]:

Выберите один ответ:

- `1:2:3:4`
- `map (<= 4) [1..]`
- `filter (<= 4) (repeat 1)`
- `1++2++3++4`
- `take 4 (iterate (+1) 1)` ✔

Вопрос **12**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию takeWhile

```
takeWhile :: (a -> Bool) -> [a] -> [a]
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

-
```
takeWhile _ [] = []
takeWhile p (x : xs)
  | p x = x : takeWhile p xs
  | otherwise = []
```
✔

-
```
takeWhile _ [] = []
takeWhile p (x : xs)
  | p x = x : takeWhile p xs
  | otherwise = takeWhile p xs
```

-
```
takeWhile p = foldl (\ acc x -> if p x then x : acc else acc)
```

-
```
takeWhile _ [] = []
takeWhile p (x : xs)
  | p x = takeWhile p xs
  | otherwise = []
```

Вопрос **13**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая объединяет список списков:

```
concat :: [[a]] -> [a]
```

Выберите один ответ:

-
```
concat [] = [[]]
concat (xs : xss) = xs ++ concat xss
```

-
```
concat [[]] = []
concat (xs : xss) = xs ++ concat xss
```

-
```
concat [] = []
concat (xs : xss) = xs ++ concat xss
```
✔

-
```
concat [] = []
concat (xs : xss) = xs : concat xss
```

Вопрос **14**

Верно

Баллов: 1,00 из 1,00

Выберите все правильные определения оператора возведения в степень ^ для неотрицательных целых чисел (в том числе 0).

Выберите один или несколько ответов:

- [x]
```
m ^ 0 = 1
m ^ n = m * m ^ (n - 1)
```
✔

- [ ]
```
m ^ 0 = 1
m ^ n = m * m * m ^ (n - 2)
```

- [ ]
```
m ^ 0 = 1
m ^ n = (m * m) ^ (n - 1)
```

- [x]
```
m ^ 0 = 1
m ^ n = m * (^) m (n - 1)
```
✔

- [ ]
```
m ^ 0 = 0
m ^ n = m * m ^ (n - 1)
```

- [ ]
```
m ^ 0 = 1
m ^ n = n * n ^ (m - 1)
```

- [ ]
```
m ^ 0 = 1
m ^ n = m * m ^ n - 1
```

- [ ]
```
m ^ 1 = m
m ^ n = m * m ^ (n - 1)
```

Вопрос **15**

Верно

Баллов: 1,00 из 1,00

Используя следующее определение, показать, как вычисляется

`length [1, 2, 3]`

```
length :: [a] -> Int
length [] = 0
length (_ : xs) = 1 + length xs
```

Выберите один ответ:

a.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (2 + length [3])
= { applying length }
1 + (2 + (3 + length []))
= { applying length }
1 + (2 + (3 + 0))
= { applying + }
6
```

b.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (1 + length [3])
= { applying length }
1 + (1 + (1 + length []))
= { applying length }
1 + (1 + (1 + 0))
= { applying + }
1 + (1 + 1)
= { applying + }
1 + 2
= { applying + }
3
```

✔

c.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (1 + length [3])
= { applying length }
1 + (1 + 1)
= { applying + }
3
```

d.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (1 + length [3])
= { applying length }
1 + (1 + (1 + length []))
= { applying length }
1 + (1 + (1 + 1))
= { applying + }
1 + (1 + 2)
= { applying + }
1 + 3
= { applying + }
4
```